ACSO - Seminarul 3

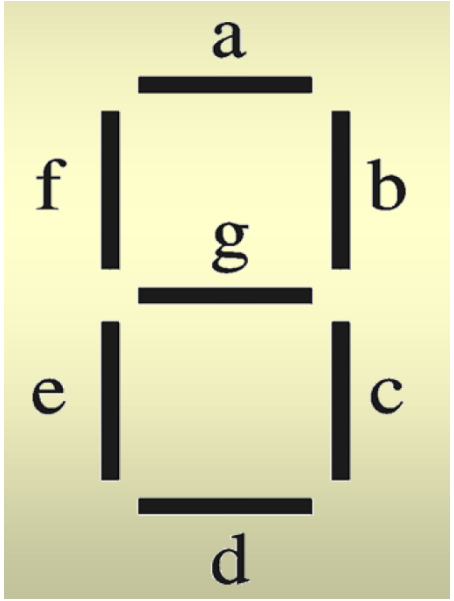
Circuite combinationale

Introducere - scurta recapitulare

- Algebra booleana, reprezentari si functiile booleene - metodologia prin care reprezentam informatia si alteram informatia (biti / octeti)
- FND - obtinerea unei ecuatii care duce la rezultatul dorit al unei functii folosind tabele logice
- Minimizarea: Simplificarea unei functii booleene pentru a scadea numarul de operatii necesare pentru a obtine rezultatul -> obtinerea unui circuit cat mai simplu - cost redus de implementare
- Exemplul practic - afisajul zecimal

Afisajul zecimal

Implementarea functiilor pentru cele 7 segmente ale afisajului:



a = F1 (A, B, C, D) = ∑m (0, 2, 3, 5, 7, 8, 9)

$$a = A + C + BD + \bar{B}\,\bar{D}$$

b = F2 (A, B, C, D) = ∑m (0, 1, 2, 3, 4, 7, 8, 9)

$$b = \bar{B} + \bar{C}\,\bar{D} + CD$$

c = F3 (A, B, C, D) = ∑m (0, 1, 3, 4, 5, 6, 7, 8, 9)

$$c = B + \bar{C} + D$$

d = F4 (A, B, C, D) = ∑m (0, 2, 3, 5, 6, 8)

$$d = \bar{B}\,\bar{D} + C\,\bar{D} + B\,\bar{C}\,D + \bar{B}\,C + A$$

e = F5 (A, B, C, D) = ∑m (0, 2, 6, 8)

$$e = \bar{B}\,\bar{D} + C\,\bar{D}$$

f = F6 (A, B, C, D) = ∑m (0, 4, 5, 6, 8, 9)

$$f = A + \bar{C}\,\bar{D} + B\,\bar{C} + B\,\bar{D}$$

g = F7 (A, B, C, D) = ∑m (2, 3, 4, 5, 6, 8, 9)

$$g = A + B\,\bar{C} + \bar{B}\,C + C\,\bar{D}$$

Cuprins

- Minimizari folosind teoreme / axiome din algebra booleana
- Set minimal de generatori
- Porti logice
- Exemple de implementari
- Circuite complexe
- Sumator
- Mux
- Decodor
- Folding

Minimizarea - teoreme / axiome

Putem folosi tabelele de axiome si teoreme din curs / seminarul 1 pentru a minimiza functii

Ex: Dorim sa minimizam urmatoarea functie: Σ(3, 5, 6, 7)

O transformam din sigma notatie in: $F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$

Aplicam idempotenta, *X + X = X*

$$F = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC + ABC + ABC$$

Aplicam unificarea *X * Y + X * (NOT Y) = X*, si obtinem forma optimizata:

$$F = BC + AC + AB$$

Set minimal de generatori

Multime de operatii binare prin care putem implementa orice functie booleana

- Datorita FND, putem deduce setul de generatori cu 3 elemente - NOT, AND, OR
- Se poate cu 2 - (NOT, AND) sau (NOT, OR) datorita legilor lui DeMorgan

A OR B = NOT ((NOT A) AND (NOT B))

Set minimal de generatori

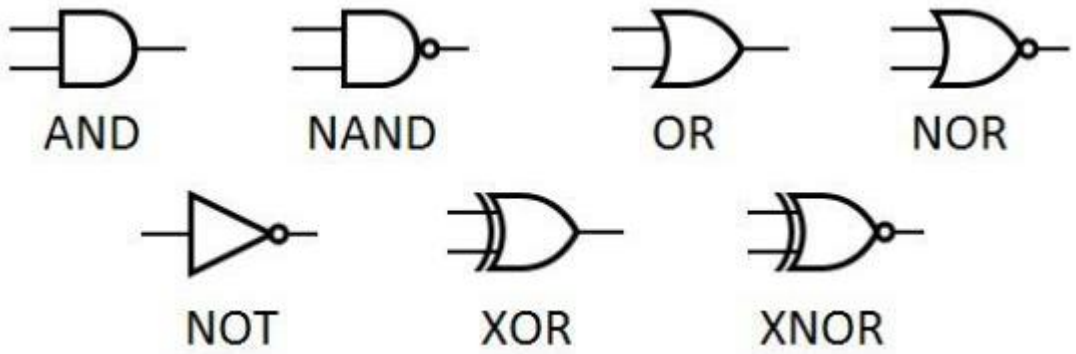
Se poate si cu o multime de 1 element: (NAND) si (NOR)

Ex: Folosind NAND pentru a reprezenta NOT, AND, OR:

- NOT (A) = A NAND A
- OR (A,B) = (A NAND A) NAND (B NAND B)
- AND (A,B) = (A NAND B) NAND (A NAND B)

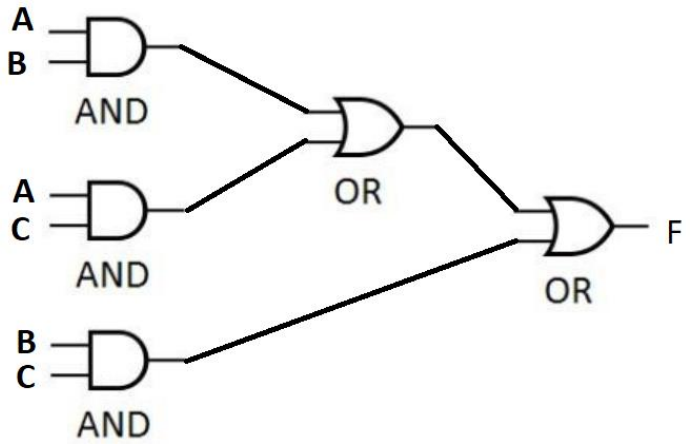
Porti logice

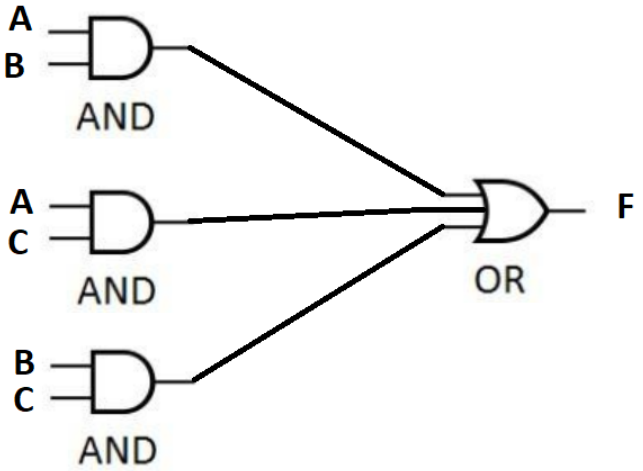
Portile logice sunt reprezentarea unor operatii booleene in contextul unui circuit



Porti logice

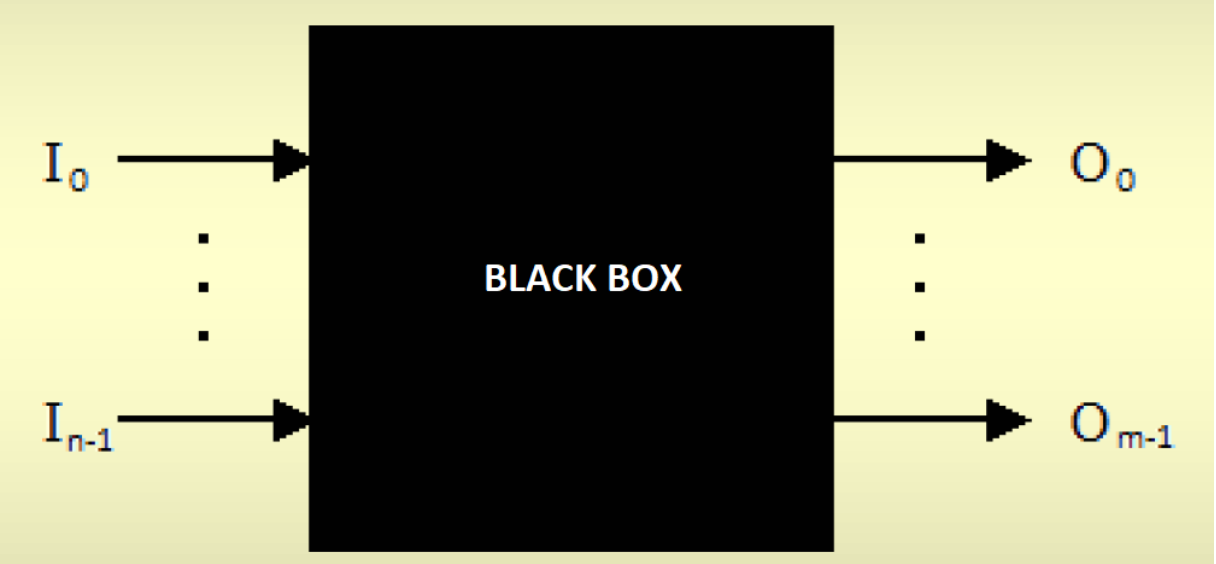
Exemplu pentru implementarea functiei: **F = BC + AC + AB**





Circuite complexe

Circuite ce corespund unei functii booleene cu N intrari si M iesiri



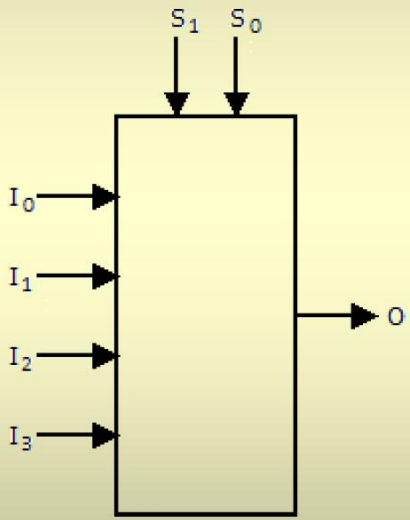
Circuite complexe - multiplexorul

Multiplexorul este un circuit complex ce are ca scop selectarea unui input pentru a fi output

- 2^n intrari
- n intrari de selectie - biti de control al outputului
- Outputul este unul din inputuri, in functie de bitii de selectie

Multiplexorul

Multiplexor 4→1 (n=2)



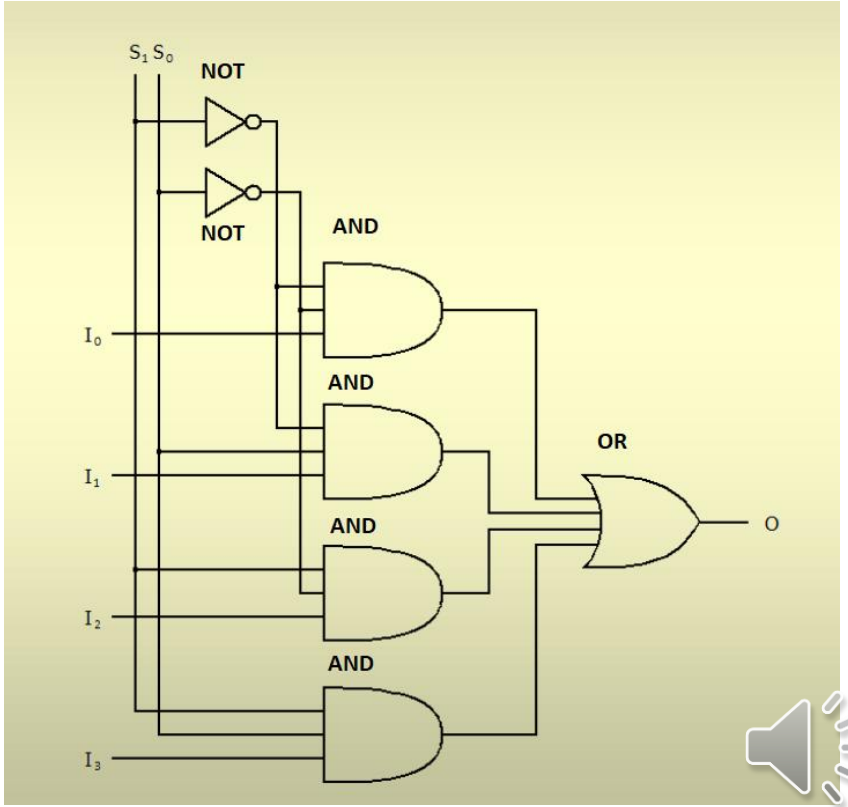
S_1	S_0	O
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Multiplexorul - implementare

- 4 porti logice AND care primesc fiecare cate un input impreuna cu bitii de selectie

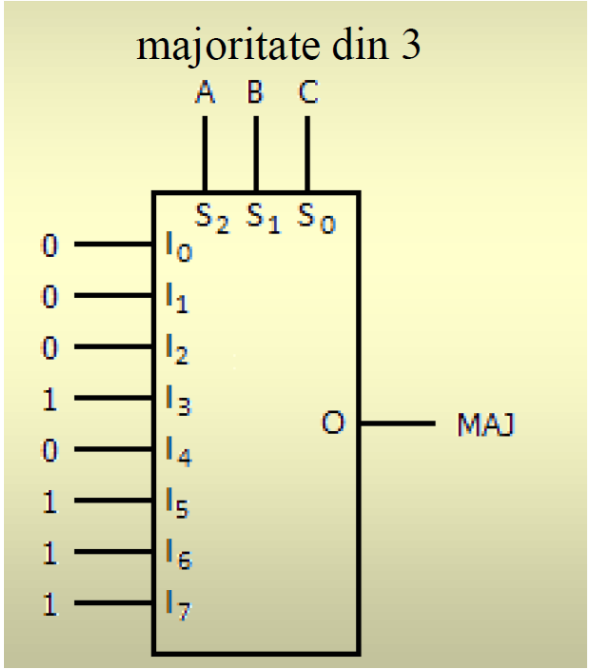
- Fiecare poarta AND returneaza True doar daca bitii de selectie pentru acea poarta ajung amandoi sa fie True si Inputul corespunzator este de asemenea True

- Cand bitii de selectie pentru o poarta AND sunt amandoi True, garantat celelalte 3 porti AND vor returna False deoarece macar unul din bitii de selectie va fi False



Multiplexor - implementarea functiilor booleene

A	B	C	
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



Exemplu de folosirea multiplexorului pentru a implementa functia majoritate din 3

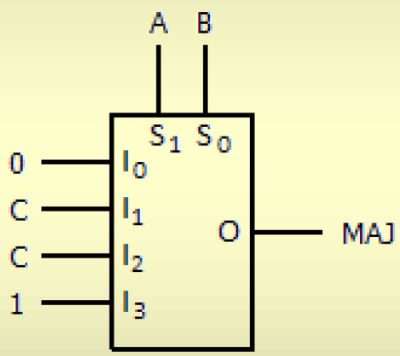
Folding - analiza inputului pentru eficientizare

Pentru anumite functii booleene, putem folosi tehnica denumita folding pentru a eficientiza circuitul multiplexorului. Analiza de caz pe majoritate din 3:

A	B	C	
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

majoritate din 3

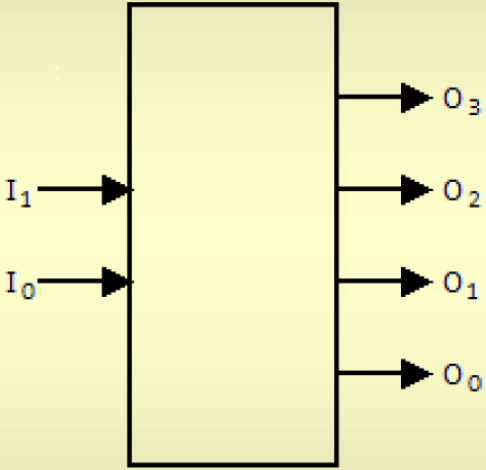
A	B	
0	0	0
0	1	C
1	0	C
1	1	1



Circuite complexe - decodorul

- N intrari
- 2^n iesiri
- Cele N intrari descriu numarul outputului care este activat pentru a avea valoarea True, restul ramanand pe False.

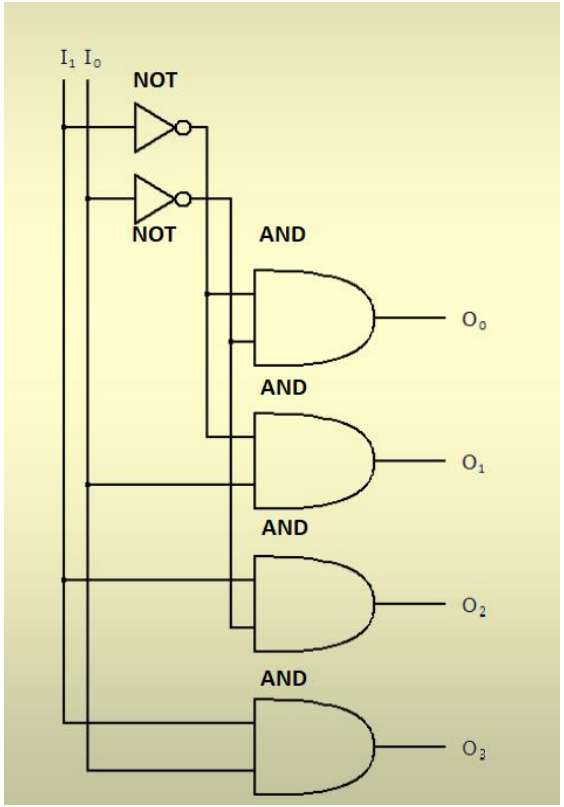
Decodorul



I_1	I_0	O_3	O_2	O_1	O_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Decodorul - implementare

- 4 porti logice AND care primesc fiecare cate o copie a inputului, alterat sau nu
- Fiecare poarta AND primeste o combinatie unica de alterare a inputului
- Daca o poarta AND are output-ul True, atunci garantat nicio alta poarta nu va avea acest output

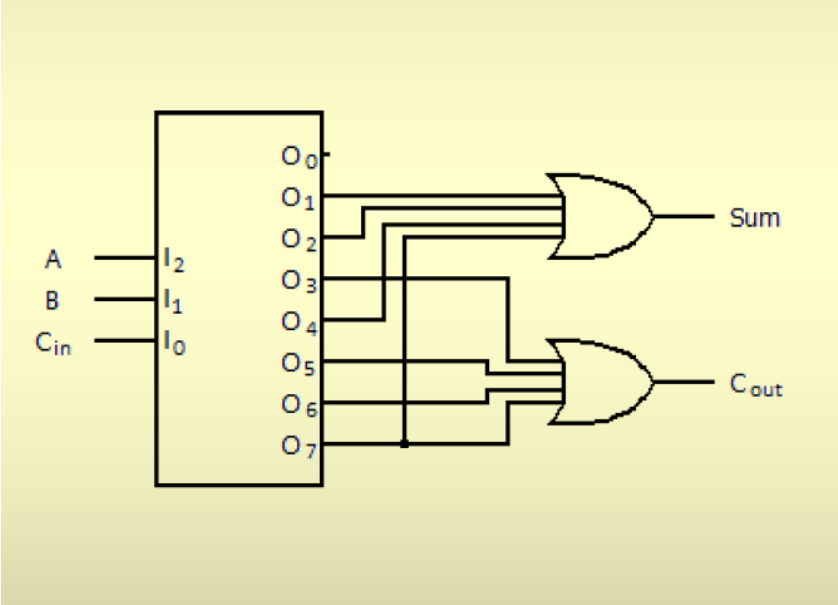


Implementarea adunarii folosind decodorul

A	B	Cin	DecOut	Sum	Cout
0	0	0	Out 0	0	0
0	0	1	Out 1	1	0
0	1	0	Out 2	1	0
0	1	1	Out 3	0	1
1	0	0	Out 4	1	0
1	0	1	Out 5	0	1
1	1	0	Out 6	0	1
1	1	1	Out 7	1	1

Implementarea adunarii folosind decodorul

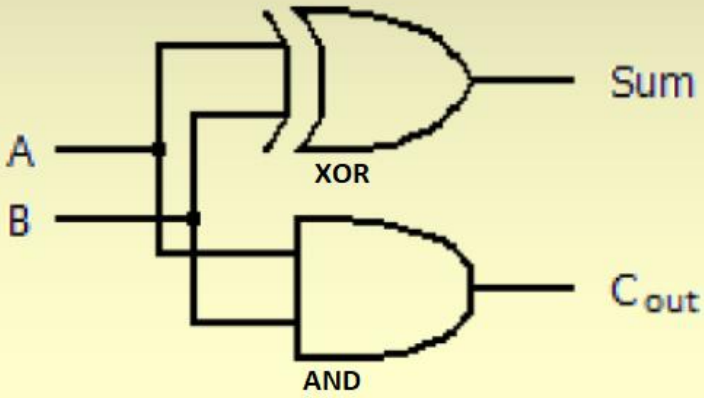
A	B	Cin	DecOut	Sum	Cout
0	0	0	Out 0	0	0
0	0	1	Out 1	1	0
0	1	0	Out 2	1	0
0	1	1	Out 3	0	1
1	0	0	Out 4	1	0
1	0	1	Out 5	0	1
1	1	0	Out 6	0	1
1	1	1	Out 7	1	1



Semi-sumatorul

Aduna 2 biti intre ei dar nu se preteaza pentru a fi aplicat unui numar format din mai multe cifre in baza 2

A	B	Sum	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



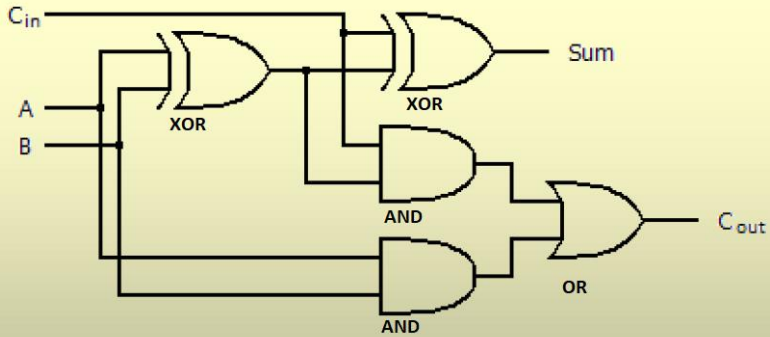
Sumatorul

Poate fi extins la mai multe cifre, si se bazeaza pe semi-sumator

A	B	Sum	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Semi-sumator

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Sumator

Exercitii

- Minimizati urmatoare functie folosind axiomele / teoremele din algebra booleana: Σ(9, 10, 11, 12, 13, 14, 15)
- Folosind NOR, determinati cele 3 formule prin care puteti construi OR, AND, NOT
- Implementati functia de la primul exercitiu folosind Multiplexorul
- Optimizati functia Imparitate folosind principiul de Folding
- Desenati diagrama aferenta circuitului pentru afisajul zecimal
- Implementati afisajul zecimal folosind un decodor